1) <u>Implementation Explanation</u>
    a. **`get_reward(self, current_state, action, next_state)`**
        i. Defines the reward signal for the agent and provides feedback based on agent's actions with its resulting state.
        ii. Goal: To incentivize desired behaviors (reaching the goal) and penalize undesirable behaviors (hitting walls)."
        iii. The reward function returns a high positive reward (100) when the agent reaches the goal state (`next_state == self.end`). It applies a negative penalty (-10) if the agent attempts to move into a wall (`self.maze[next_state[0], next_state[1]] == 1`). Otherwise, a small negative reward (-0.1) is given for each step, encouraging the agent to find the shortest path."
    b. **`is_valid_state(self, state)`**
        i. Ensures that the agent stays within the boundaries of the maze. It prevents the agent from taking actions that would lead it outside the maze.
        ii. The function checks if the row (y) and column (x) indices of the given `state` are within the valid range (0 to `self.size` - 1). It returns `True` if the state is within bounds and `False` otherwise.
    c. **`decide_action(self, state, q_table, epsilon)`**
        i. Implementation of the epsilon-greedy exploration strategy. It balances the agent's desire to explore new actions with its desire to exploit its current knowledge.
        ii. With a probability of `epsilon`, the agent chooses a random action from the set of possible actions (Up, Right, Down, Left), promoting exploration. Otherwise, the agent chooses the action with the highest Q-value in the current state, according to the Q-table (`np.argmax(q_table[state[0], state[1]])`), promoting exploitation. If multiple actions have the same maximum Q-value, it chooses randomly among them to avoid bias.
    d. **`handle_movement(self, current_state, action)`**
        i. Simulates the agent's movement within the maze. It takes the current state and chosen action as input, calculates the next state, checks for wall collisions, and returns the next state, reward, and cost increment.
        ii. It calculates the next state based on the chosen action. If the next state is out of bounds or if it is a wall, the agent stays in the current state, and the function indicates that there is cost using

'increment_cost'. The appropriate reward is given based on whether there is a termination point based on the final state.

e. **`q_learning(self, num_episodes=100, learning_rate=0.3, discount_factor=0.9, exploration_rate=0.5, delay=3)`**
    i. The core of the Q-Learning algorithm. It trains the agent to find the optimal path through the maze.
    ii. The algorithm iterates through a specified number of episodes (`num_episodes`). In each episode, the agent starts at the starting position and takes actions until it reaches the goal, or a maximum number of steps is reached. The agent chooses actions using the epsilon-greedy strategy (`choose_action`).
    iii. The agent's Q-table is updated using the Q-Learning update rule (`q_table[state[0], state[1], action] += learning_rate * temporal_difference`). The `handle_movement` function is used to handle the states, check for the values, and apply the values during the Q-learning stage. As the equation approaches, the program converges to the shortest route possible, or optimal settings.

f. **`q_learning_path(self, q_table)`**
    i. After the Q-Learning algorithm has been trained, this function extracts the learned path from the Q-table.
    ii. Starting from the initial state, the function iteratively selects the action with the highest Q-value in the current state. The `handle_movement` function is then used to take the intended action and move into the next state and returns data back, for analysis. This process continues until the goal state is reached, the final path and the total code are returned.

2) <u>Demonstration of Algorithm's Performance</u>

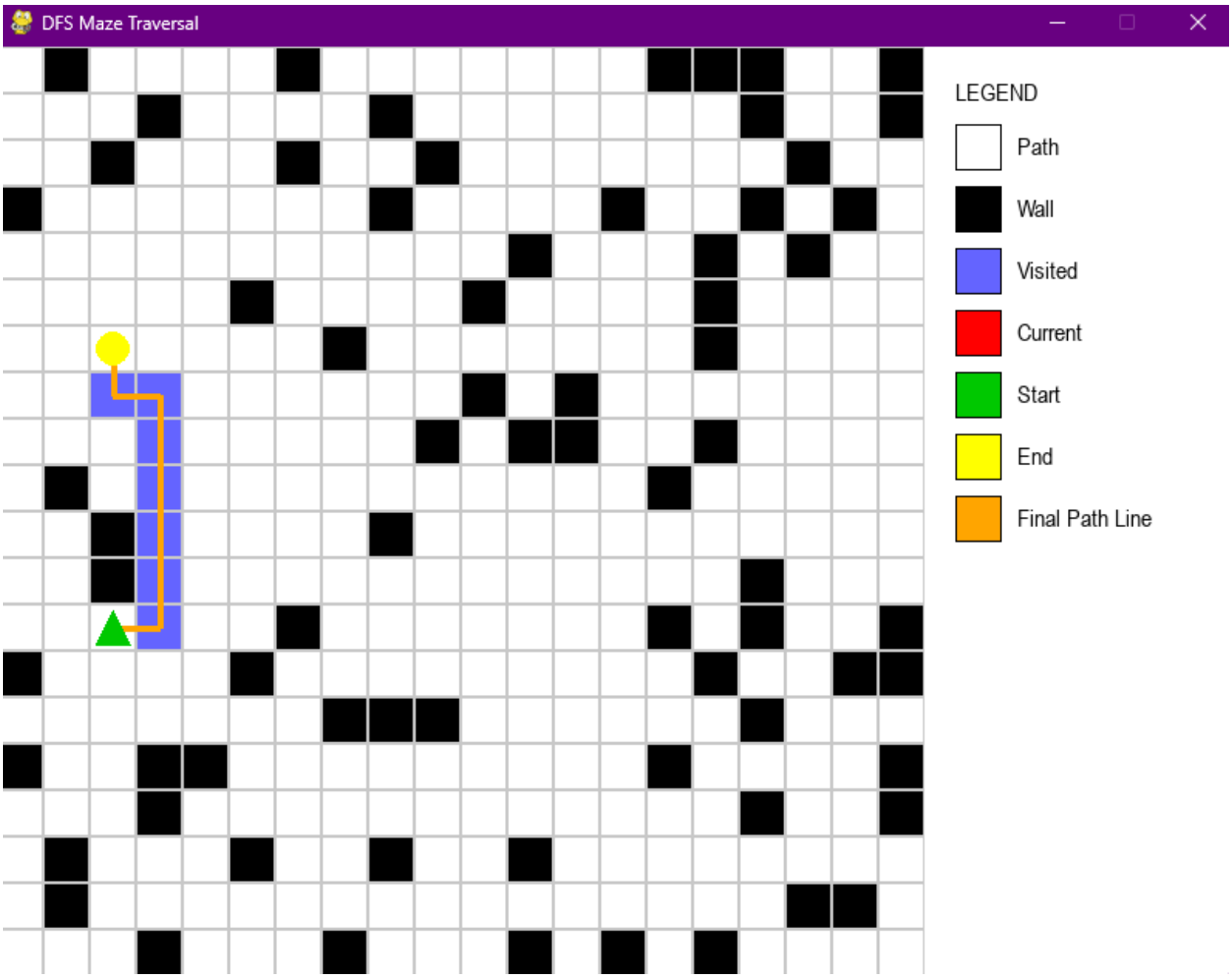

Figure 1. (WP=0.2, LR=0.6, DF=0.8)

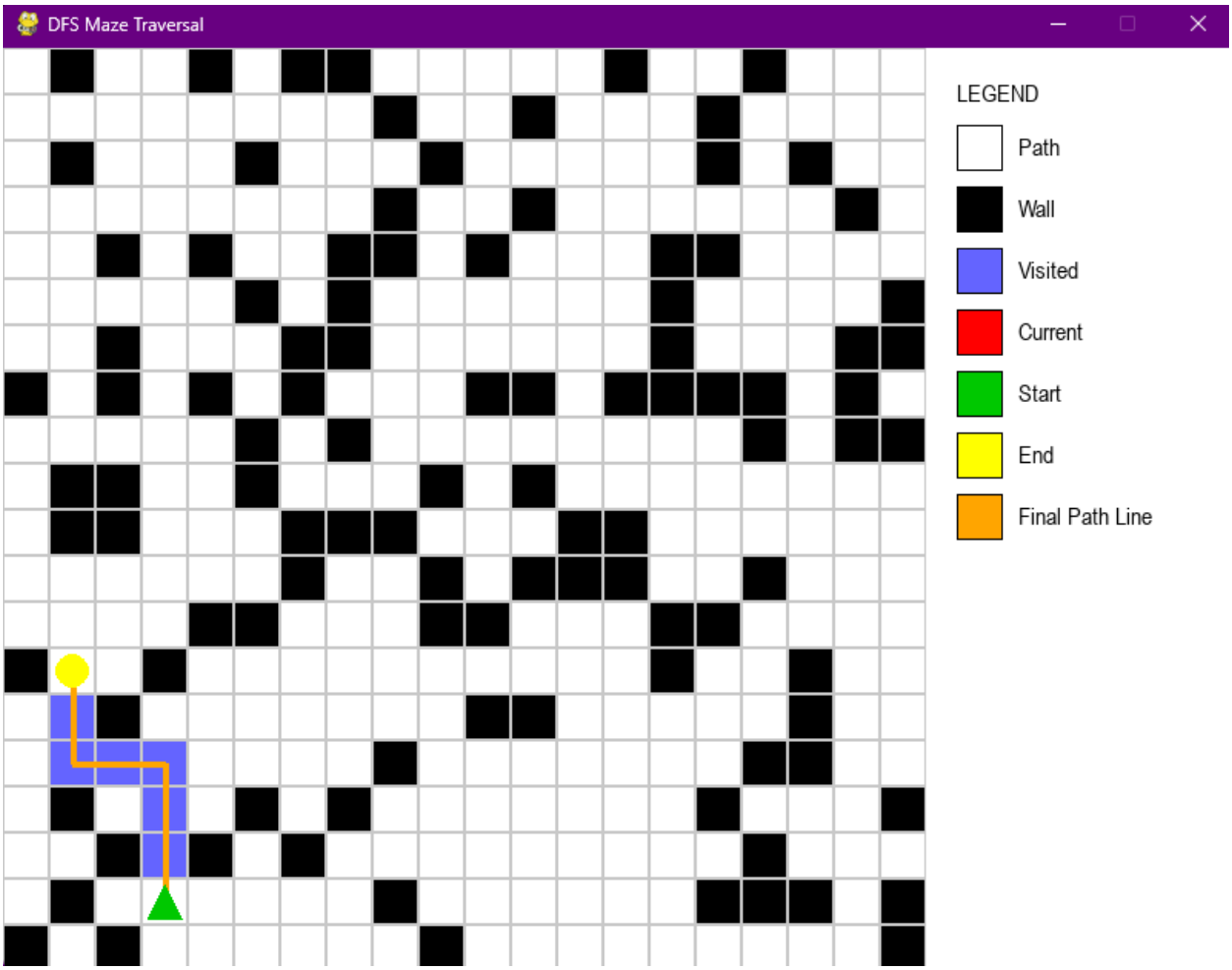

Figure 2. (WP=0.3, LR=0.6, DF=0.8)

a. To evaluate the robustness of the Q-Learning algorithm, tests were conducted on three randomly generated mazes with varying wall probabilities (WP = 0.2, 0.3, and 0.4). In all tests, the Learning Rate (α) and Discount Factor (γ) were held constant at 0.6 and 0.8, respectively. The performance of the algorithm was assessed based on its ability to find the shortest path from the starting position to the goal, as visualized using the provided visualization functions.
b. Observation:
    i. WP = 0.2 and WP = 0.3: The Q-Learning algorithm successfully identified the shortest path in both mazes with wall probabilities of 0.2 and 0.3. The visualized paths clearly demonstrated efficient navigation through the maze, indicating that the chosen parameter

combination (α = 0.6, γ = 0.8) was sufficient for learning an optimal policy in these less complex environments.

ii. WP = 0.4: The algorithm failed to find a path in the maze with a wall probability of 0.4. The visualization output reported "no path may exists. please change the random seed or size.", suggesting that the maze was either too complex for the current parameter settings or that, by random chance, a viable path between the start and goal simply did not exist in the generated maze.

c. Interpretation:

i. A higher wall probability (WP = 0.4) translates to a more challenging path-finding problem. As the number of walls increases, the length of the shortest path is likely to increase, and the number of possible paths (both optimal and suboptimal) is likely to decrease. With these settings, the algorithm fails.

ii. The fact that Q-Learning can solve for 0.2 and 0.3 but not for 0.4 might indicates that the combination can only solve up to certain limit. Because a 0.4 wall probability means a higher complexity in the maze itself, a higher learning rate may be required to update the path quicker and to allow the agent to explore and learn the best possible path, or one with larger discount rate that makes the agent put emphasis on planning.

3) <u>Impact of Parameters</u>

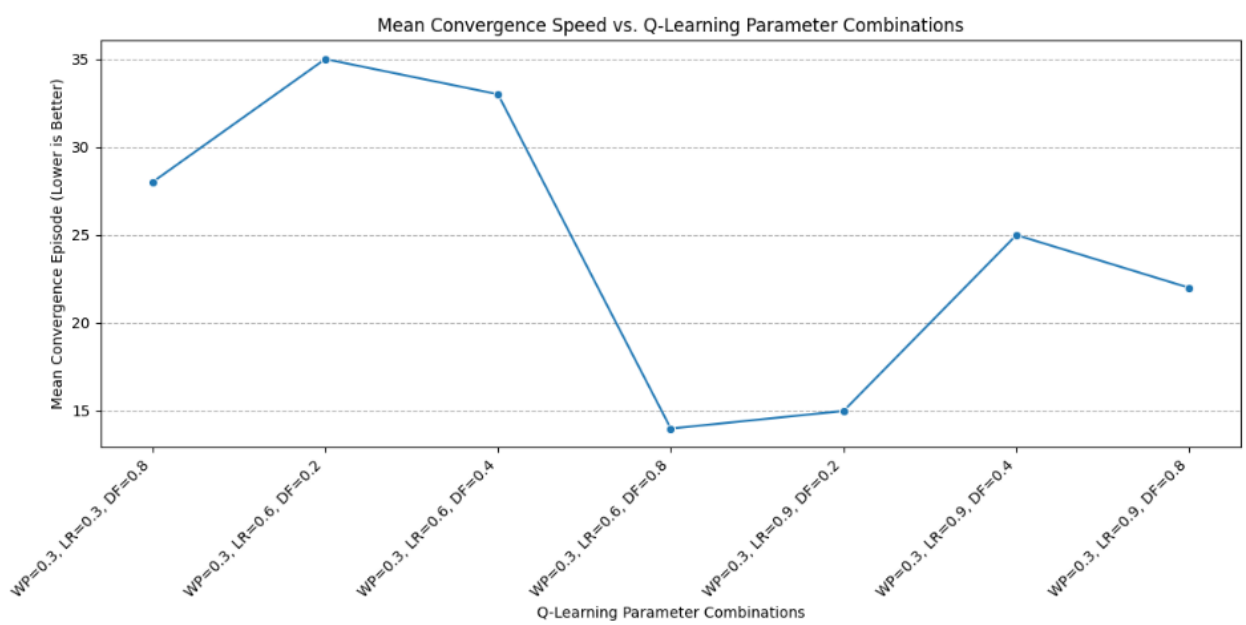

Figure 3

```
Summary Statistics:
   wall_prob  learning_rate  discount_factor  mean_path_cost  std_path_cost  \
0        0.3            0.3              0.8            22.0            NaN
1        0.3            0.6              0.2            22.0            NaN
2        0.3            0.6              0.4            22.0            NaN
3        0.3            0.6              0.8            22.0            NaN
4        0.3            0.9              0.2            22.0            NaN
5        0.3            0.9              0.4            22.0            NaN
6        0.3            0.9              0.8            22.0            NaN

   count  mean_total_reward  mean_convergence_episodes  \
0      1             8103.6                       28.0
1      1             8061.0                       35.0
2      1             8241.4                       33.0
3      1             8553.3                       14.0
4      1             8744.4                       15.0
5      1             8907.1                       25.0
6      1             8793.4                       22.0

   median_convergence_episodes
0                         28.0
1                         35.0
2                         33.0
3                         14.0
4                         15.0
5                         25.0
6                         22.0
```

Figure 4

a. Settings:
    i. WP (Wall Probability): This parameter is held constant at 0.3 across all experiments.
    ii. LR (Learning Rate, α): Values of 0.3, 0.6, and 0.9 were tested.
    iii. DF (Discount Factor, γ): Values of 0.2, 0.4, and 0.8 were tested.
    iv. Some combinations may not be included since these combinations could not result in finding path within 100 iterations.
b. Analysis of Result:
    i. Low Learning Rate and High Discount Factor (WP=0.3, LR=0.3, DF=0.8): The experiment with a low learning rate (0.3) and a high discount factor (0.8) required approximately 28 episodes to converge. This suggests that while a low learning rate promotes stability and a

high discount factor encourages long-term planning, this combination may be somewhat slow to adapt to the maze structure.

ii. Higher Learning Rate with Lower Discount Factor (WP=0.3, LR=0.6, DF=0.2 and DF=0.4): Increasing the learning rate to 0.6 while using discount factors of 0.2 and 0.4 resulted in longer convergence times (approximately 35 and 33 episodes, respectively). This is likely because a maze with wall probability 0.3 requires the agent to plan several steps ahead, the smaller discount factor diminishes the importance of planning, increasing the convergence time.

iii. Low Learning Rate with High Discount Factor (WP=0.3, LR=0.6, DF=0.8): Requiring only approximately 14 episodes. It is efficient to have a smaller learning rate while valuing long term rewards over short term ones.

iv. High Learning Rate with Lower Discount Factor (WP=0.3, LR=0.9, DF=0.2 and DF=0.4): The experiment with the higher learning rate (0.9) and lower discount factors converge between 15 and 25 episodes.

v. High Learning Rate with High Discount Factor (WP=0.3, LR=0.9, DF=0.8): Requiring only approximately 22 episodes to converge.

c. Conclusion:

i. The results show that the learning rate is not simply a matter of "faster is always better." A high learning rate (0.9) without a compensating strategic outlook (high γ) did not consistently achieve the best results. The learning rate must be tuned to the characteristics of the reward and environment.

ii. The discount factor's influence is more varied. It's clear that a low discount factor makes it harder for the agent to navigate longer, more complex mazes effectively. However, when the learning rate is high enough (i.e., immediate experiences are rapidly incorporated), a moderate discount factor may suffice.